

TreeFormer 系の非連結植物骨格推定に向けた 仮想ルートノード付き森林制約の定式化

理論整理・実装設計メモ

2026 年 7 月 9 日

Abstract

本資料は、TreeFormer/PlantPose 系の「画像から植物骨格グラフを推定する」問題を、単一連結木ではなく、画像内に複数の独立した木が存在し得る**森林推定問題**として定式化し直す。中心となる提案は、実ノード集合に**仮想ルートノード**、すなわち **dummy root / super-root** を追加し、拡張グラフ上で全域木を解いた後、仮想ルートに接続する edge を取り除くことで実ノード上の森林を得る方法である。この定式化により、単一 MST が本来非連結である複数物体間に **bridge edge** を作る問題を回避できる。さらに、TreeFormer の Selective Feature Suppression (SFS) を、仮想ルート付き MST/SFS、rooted forest CRF、または **perturbed spanning forest loss** へ拡張するための数式、損失関数、推論手順、実装契約、検証項目を自己完結的に整理する。

Contents

1	文書の目的と結論	4
1.1	目的	4
1.2	主要結論	4
1.3	推奨する実装順	4
2	背景: TreeFormer/PlantPose の単一 MST 制約	5
3	単一 MST が破綻する条件	5
3.1	単一木制約	5
3.2	複数物体に対する矛盾	5
4	仮想ルート付き森林定式化	6
4.1	基本定義	6
4.2	仮想ルート付き MST から森林への射影	6
4.3	概念図	7
5	スコア設計	7
5.1	実 edge score	7
5.2	root edge score	7
5.3	component 開始ペナルティ	7

6 教師データのケース分け	8
6.1 ケース A: component ID がある	8
6.2 ケース B: component ID はないが GT edge がある	8
6.3 ケース C: node だけあり edge がない	8
6.4 ケース D: 遮蔽による見かけ上の非連結	8
7 node matching と edge label の定義	9
8 基本 loss 群	9
8.1 node loss	9
8.2 edge BCE/CE	9
8.3 cross-component negative loss	9
8.4 root loss: 代表 root が指定される場合	10
8.5 root loss: 代表 root が任意の場合	10
8.6 global component count loss	10
9 Virtual-Root MST/SFS loss	10
9.1 拡張スコア行列	10
9.2 projection mask	11
9.3 SFS の一般形	11
9.4 Virtual-Root SFS loss	11
10 Rooted Forest CRF: より上位互換に近い定式化	12
10.1 有向仮想ルートグラフ	12
10.2 Matrix-Tree partition	12
10.3 root 固定の NLL	12
10.4 root 未指定の場合の周辺化	13
10.5 Matrix-Forest との関係	13
11 Perturbed Spanning Forest loss	13
12 GNN/adjacency loss との関係	14
12.1 GNN loss でできること	14
12.2 GNN loss でできないこと	14
12.3 推奨する併用	14
13 推論アルゴリズム	14
13.1 component 数が未知の場合	14
13.2 component 数が既知の場合	15
13.3 node 数も未知の場合	15

14 ONNX/OpenVINO を見据えた出力契約	15
15 実装仕様案	16
15.1 forward 出力	16
15.2 postprocessor 出力	16
15.3 loss module 入力	16
15.4 実装上の重要パラメータ	16
16 評価指標	17
17 想定される失敗と対処	18
17.1 root edge が安すぎる場合	18
17.2 root edge が高すぎる場合	18
17.3 root が任意なのに固定 root CE を使う場合	18
17.4 遮蔽と別個体を混同する場合	18
18 実装検証用の最小テスト	18
19 研究候補の比較	19
20 最終提案	19
21 補足: 最小実装の疑似手順	20
21.1 訓練時	20
21.2 推論時	20
22 結語	20

1 文書の目的と結論

1.1 目的

本資料の目的は、以下の問いに対して、他の実装者が迷わず設計・実装・検証できる程度に、自己完結した定式化を与えることである。

1. TreeFormer の単一 MST 制約を、画像内に複数の独立した木が存在する場合へどう拡張するか。
2. 仮想ルートノードを追加すると、なぜ森林が得られるのか。
3. root edge、実 edge、component 数、cross-component negative をどのように loss 化するか。
4. GNN 的な edge BCE/CE だけでは不十分な理由と、上位互換に近い structured loss 候補をどう整理するか。
5. ONNX/OpenVINO 等のデプロイを見据えたとき、モデル出力とグラフ後処理の境界をどう切るか。

1.2 主要結論

結論は以下である。

1. 単一木を仮定した MST は、画像中に複数の別個体がある場合、本来存在しない component 間 bridge edge を必ず 1 本以上作る。これは問題設定と矛盾する。
2. 実ノード集合 V に仮想ルート 0 を加えた拡張グラフ $G^+ = (V \cup \{0\}, E^+)$ 上で全域木 T^+ を解き、root edge を削除すると、実ノード上に森林 F が得られる。
3. T^+ における仮想ルートの次数 $\deg_{T^+}(0)$ は、root edge 削除後の森林の component 数と一致する。
4. したがって、非連結物体対応の最小拡張は、TreeFormer の MST/SFS を **Virtual-Root MST/SFS** へ置き換えることである。
5. 研究的には、仮想ルート付き有向全域木 CRF、すなわち Rooted Forest CRF を使うと、root 候補を周辺化しつつ森林全体の尤度を直接最大化できる。
6. GNN や単純な edge BCE は edge score 改善には有効だが、非連結性、無閉路性、component 数、must-not-connect 制約を直接保証しない。主 loss ではなく補助 loss と見るべきである。

1.3 推奨する実装順

短期実装では以下を推奨する。

$$\mathcal{L}_{total} = \lambda_{node}\mathcal{L}_{node} + \lambda_{edge}\mathcal{L}_{edge} + \lambda_{root}\mathcal{L}_{root} + \lambda_{cross}\mathcal{L}_{cross} + \lambda_{vr}\mathcal{L}_{VR-SFS} \quad (1)$$

ここで $\mathcal{L}_{VR-SFS}$ は、仮想ルート付き MST に基づく SFS loss である。

中期研究実装では、以下を比較対象にする。

$$\mathcal{L}_{VR-CRF} = -\log P(F^* | I), \quad (2)$$

$$\mathcal{L}_{PerturbedForest} = \text{partial Fenchel-Young / perturb-and-MSF loss}, \quad (3)$$

$$\mathcal{L}_{OT-Graph} = \text{graph-level OT loss as auxiliary loss}. \quad (4)$$

2 背景: TreeFormer/PlantPose の単一 MST 制約

TreeFormer は、画像から植物骨格を任意トポロジの木グラフとして推定するため、学習型 graph generator と MST 等の伝統的グラフアルゴリズムを組み合わせる。論文では、tree graph generation は edge 集合全体に対する制約を満たす必要があるため組合せ最適化になると説明されている。また、単純な後処理 MST だけでは、学習時の generator が制約を見ないため出力が不自然になり得る。そこで、MST projection を訓練ループに入れ、SFS layer により非微分な MST の効果を微分可能に模倣する、という設計を採っている [1, 2]。

PlantPose は TreeFormer の拡張版として、同じ SFS/MST の思想をより大規模・汎用的な植物骨格推定へ拡張している。PlantPose の説明でも、MST projection と SFS により、unconstrained graph generator の edge 予測を tree constraint へ整合させることが中核とされている [2]。

ただし、既存の TreeFormer/PlantPose の中核仮定は「対象グラフは単一連結木である」という点にある。画像内に 2 本の別個の木が写っている場合、この仮定は破綻する。単一 MST は全 node を 1 つの連結 component にするため、物理的に別個体である 2 本の木の間にも bridge edge を作ってしまう。

3 単一 MST が破綻する条件

3.1 単一木制約

通常の木制約は、node 集合 V に対して edge 集合 E が以下を満たすことを要求する。

$$|E| = |V| - 1, \quad (5)$$

$$(V, E) \text{ is connected}, \quad (6)$$

$$(V, E) \text{ is acyclic}. \quad (7)$$

この 3 条件は同値的に「 $G = (V, E)$ が spanning tree である」ことを意味する。

3.2 複数物体に対する矛盾

画像中に 2 本の別個の木 $G_1 = (V_1, E_1)$ と $G_2 = (V_2, E_2)$ があるとする。真の構造は

$$V = V_1 \cup V_2, \quad E = E_1 \cup E_2, \quad V_1 \cap V_2 = \emptyset \quad (8)$$

であり、 G は 2 component の森林である。したがって

$$|E| = (|V_1| - 1) + (|V_2| - 1) = |V| - 2. \quad (9)$$

一方、単一 MST は $|V| - 1$ 本の edge を要求する。差分の 1 本は、必ず V_1 と V_2 を接続する bridge edge になる。

$$\underbrace{|V| - 1}_{\text{single MST}} - \underbrace{(|V| - 2)}_{\text{2-component forest}} = 1. \quad (10)$$

この 1 本は真の edge ではない。したがって、複数物体を許す場合、制約は tree ではなく forest に変更する必要がある。

4 仮想ルート付き森林定式化

4.1 基本定義

定義 1 (実ノード集合). 画像から検出される実ノード集合を

$$V = \{1, 2, \dots, N\} \quad (11)$$

とする。最大候補数 K を持つ *DETR/RelationFormer* 型モデルでは、推論後に有効 *node* 集合 $A \subseteq \{1, \dots, K\}$ を選び、ここでは $|A| = N$ として議論する。

定義 2 (仮想ルート付き拡張ノード集合). 仮想ルートノード 0 を追加し、

$$V^+ = V \cup \{0\} \quad (12)$$

と定義する。 0 は画像中の物理的な点ではなく、複数 *component* を束ねるための計算上の *dummy root* である。

定義 3 (拡張 *edge* 集合). 実ノード間 *edge* 候補を $E \subseteq \{(i, j) : i, j \in V, i < j\}$ とする。仮想ルートから各実ノードへの *root edge* を追加し、

$$E^+ = E \cup E_0, \quad E_0 = \{(0, i) : i \in V\} \quad (13)$$

とする。

4.2 仮想ルート付き MST から森林への射影

拡張グラフ $G^+ = (V^+, E^+)$ 上で全域木 T^+ を解く。

$$T^* = \arg \max_{T^+ \in \mathcal{T}(G^+)} S(T^+), \quad (14)$$

ここで $\mathcal{T}(G^+)$ は G^+ 上の全域木集合である。スコアは後述する。得られた T^+ から *root edge* を除去し、実ノード上の予測森林を得る。

$$F = T^+ \setminus E_0. \quad (15)$$

命題 1 (仮想ルート除去により森林が得られる). T^+ を V^+ 上の任意の木とする。 $F = T^+ \setminus E_0$ は実ノード集合 V 上の森林である。

Proof. T^+ は木なので閉路を持たない。 F は T^+ の部分グラフであるから、 F も閉路を持たない。閉路を持たない無向グラフは森林である。したがって F は V 上の森林である。□

命題 2 (仮想ルート次数と *component* 数). T^+ を V^+ 上の木、 $F = T^+ \setminus E_0$ とする。このとき F の連結成分数は、 T^+ における仮想ルートの次数と一致する。

$$\kappa(F) = \deg_{T^+}(0). \quad (16)$$

Proof. 木 T^+ から頂点 0 とそれに接続する $\deg_{T^+}(0)$ 本の *edge* を削除する。木から次数 d の頂点を削除すると、残りのグラフは d 個の連結成分に分かれる。ここで $d = \deg_{T^+}(0)$ である。残りのグラフは F なので、 $\kappa(F) = d$ である。□

命題 3 (森林から仮想ルート付き木への拡張). $F = (V, E_F)$ を C 個の連結成分 $C_1, \dots, C_C$ を持つ森林とする。各成分から代表 $noder_c \in C_c$ を 1 つ選び、

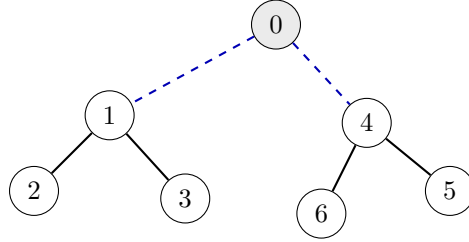
$$T^+ = F \cup \{(0, r_c) : c = 1, \dots, C\} \quad (17)$$

と定義すると、 T^+ は V^+ 上の木である。

Proof. F は C 個の木の非交和である。各木に対して仮想ルート 0 から 1 本ずつ edge を追加すると、全 component は 0 を介して連結になる。追加 edge は各 component に 1 本だけなので閉路は生じない。よって T^+ は連結かつ無閉路、すなわち木である。□

以上により、仮想ルート付き木と **root** 付き森林は、root edge を除去する操作を通じて自然に対応する。

4.3 概念図



拡張木 T^+ から青い root edge を除去すると、左の木と右の木からなる森林 F が残る。

5 スコア設計

5.1 実 edge score

モデルの node token 特徴を $h_i \in \mathbb{R}^d$ とする。実ノード間の edge logit を

$$s_{ij} = f_{\text{edge}}(h_i, h_j, \phi_{ij}) \quad (18)$$

とする。 ϕ_{ij} は相対座標、距離、方向、画像特徴などの pair feature である。無向 edge であれば

$$s_{ij} = s_{ji} \quad (19)$$

にするか、双方向スコアを平均する。

$$s_{ij}^{\text{sym}} = \frac{1}{2}(s_{i \rightarrow j} + s_{j \rightarrow i}). \quad (20)$$

5.2 root edge score

仮想ルートから実 node i への root edge score を

$$r_i = f_{\text{root}}(h_i, \psi_i) \quad (21)$$

とする。 r_i は、「node i がある component の代表 root として適切か」を表す。物理的な根元が明確な植物であれば根元らしさ、単に component 代表でよい場合は component 開始点らしさを学習する。

5.3 component 開始ペナルティ

component 数を制御するため、root edge にペナルティ $\lambda \geq 0$ を入れる。

$$S(T^+) = \sum_{(i,j) \in T^+, i,j \neq 0} s_{ij} + \sum_{(0,i) \in T^+} (r_i - \lambda). \quad (22)$$

式(16)より、 λ は component 数に対する線形ペナルティである。

$$S(T^+) = \sum_{(i,j) \in F} s_{ij} + \sum_{c=1}^{\kappa(F)} r_{\rho_c} - \lambda \kappa(F), \quad (23)$$

ここで ρ_c は component c の root として選ばれた node である。

直観は以下である。 λ を大きくすると component 開始が高コストになり、component 数は少なくなりやすい。反対に、 λ を小さくすると component 開始が低コストになり、component 数は多くなりやすい。ただし、実際の最適解は実 edge score と root score の相対値にも依存するため、 λ は validation で較正する。

6 教師データのケース分け

6.1 ケース A: component ID がある

各 GT node $u \in V^*$ に component label $c(u) \in \{1, \dots, C^*\}$ がある場合が最も扱いやすい。この場合、cross-component negative を明示的に作れる。

$$c(u) \neq c(v) \Rightarrow (u, v) \notin E^*. \quad (24)$$

6.2 ケース B: component ID はないが GT edge がある

GT edge 集合 E^* があれば、GT graph の connected components を計算することで component ID を復元できる。

$$\{C_1^*, \dots, C_{C^*}^*\} = \text{CC}(V^*, E^*). \quad (25)$$

この場合も、異なる GT connected component 間を cross-component negative として扱える。

6.3 ケース C: node だけあり edge がない

node だけの場合、森林構造の教師はない。この場合、仮想ルート付き森林 loss は直接使えない。代替として、距離・方向・外観に基づく擬似 edge、annotated scribble、または teacher model の pseudo graph を作る必要がある。

6.4 ケース D: 遮蔽による見かけ上の非連結

重要なのは、別個体による非連結と遮蔽・検出漏れによる見かけ上の非連結を分けることである。別個体なら forest が正しい。

$$\text{multiple physical instances} \Rightarrow \text{forest} \quad (26)$$

遮蔽された 1 本の木なら、latent edge または latent node を考えるべきである。

$$\text{occluded single instance} \Rightarrow \text{latent connection or hidden node} \quad (27)$$

この区別をアノテーション仕様で定めないと、モデルは「切るべき非連結」と「補完すべき非連結」を混同する。

7 node matching と edge label の定義

DETR 系モデルでは、予測 node 候補は固定長 K が出る。GT node 集合 V^* との対応は Hungarian matching で決める。

$$\pi : V^* \rightarrow \{1, \dots, K\}. \quad (28)$$

対応済み予測 node 集合を

$$A = \pi(V^*) \quad (29)$$

とする。GT edge label は

$$y_{ij}^* = \mathbf{1} [\exists (u, v) \in E^* \text{ s.t. } i = \pi(u), j = \pi(v)] \quad (30)$$

である。component label は

$$g_i^* = c(u) \quad \text{if } i = \pi(u) \quad (31)$$

と定義する。異なる component の pair 集合を

$$\mathcal{P}_{cross} = \{(i, j) : i, j \in A, g_i^* \neq g_j^*\} \quad (32)$$

とする。同一 component だが直接 edge ではない pair も存在するため、

$$g_i^* = g_j^* \nRightarrow y_{ij}^* = 1 \quad (33)$$

に注意する。

8 基本 loss 群

8.1 node loss

node 検出 loss は既存 TreeFormer/RelationFormer 型を維持する。

$$\mathcal{L}_{node} = \mathcal{L}_{cls} + \lambda_{coord} \mathcal{L}_{coord} + \lambda_{box} \mathcal{L}_{box}. \quad (34)$$

植物骨格 node だけを扱うなら、box loss を省略し、座標 loss を中心にしてよい。

8.2 edge BCE/CE

実 edge に対する基本 loss は、class imbalance を考慮した BCE または CE である。

$$\mathcal{L}_{edge} = -\frac{1}{|\mathcal{P}|} \sum_{(i,j) \in \mathcal{P}} [\alpha y_{ij}^* \log p_{ij} + (1 - y_{ij}^*) \log(1 - p_{ij})], \quad (35)$$

ここで $p_{ij} = \text{sigmoid}(s_{ij})$ である。 $\alpha > 1$ は正 edge が少ないことへの重みである。

8.3 cross-component negative loss

異なる物体間に bridge edge を張らないため、cross-component negative を明示的に入れる。

$$\mathcal{L}_{cross} = -\frac{1}{|\mathcal{P}_{cross}|} \sum_{(i,j) \in \mathcal{P}_{cross}} \log(1 - p_{ij}). \quad (36)$$

この loss は、仮想ルート方式において特に重要である。root edge が正しく較正されていない初期学習では、別 component 間の実 edge score が高くなり、単一木的な bridge が選ばれやすいためである。

8.4 root loss: 代表 root が指定される場合

各 component に正解 root node r_c^* がある場合、root edge の分類 loss を使う。

$$y_{0i}^* = \mathbf{1}[i = \pi(r_c^*) \text{ for some } c]. \quad (37)$$

$$\mathcal{L}_{root}^{BCE} = -\frac{1}{|A|} \sum_{i \in A} [y_{0i}^* \log p_{0i} + (1 - y_{0i}^*) \log(1 - p_{0i})]. \quad (38)$$

ここで $p_{0i} = \text{sigmoid}(r_i)$ である。

8.5 root loss: 代表 root が任意の場合

component 内のどの node を root にしてもよい場合、特定 node を正解 root に固定すると不必要なバイアスになる。この場合、multiple-instance learning (MIL) 型の loss を使う。

$$\mathcal{L}_{root}^{MIL} = -\sum_{c=1}^{C^*} \log \left(1 - \prod_{i \in A_c} (1 - p_{0i}) \right), \quad (39)$$

ここで $A_c = \pi(V_c^*)$ である。これは「各 component 内で少なくとも 1 つ root edge が高くなればよい」という loss である。

ただし、これだけでは component 内で複数 root が高くなる可能性がある。そこで count penalty を追加する。

$$\mathcal{L}_{root}^{count} = \sum_{c=1}^{C^*} \left(\sum_{i \in A_c} p_{0i} - 1 \right)^2. \quad (40)$$

最終的な root loss は

$$\mathcal{L}_{root} = \mathcal{L}_{root}^{MIL} + \beta \mathcal{L}_{root}^{count} \quad (41)$$

とする。

8.6 global component count loss

画像ごとの component 数 C^* がわかる場合、root probability の総和にも制約をかける。

$$\mathcal{L}_{global-count} = \left(\sum_{i \in A} p_{0i} - C^* \right)^2. \quad (42)$$

別 head で component count を分類する場合は、

$$\mathcal{L}_{count} = \text{CE}(\hat{C}, C^*) \quad (43)$$

を追加する。

9 Virtual-Root MST/SFS loss

9.1 拡張スコア行列

対応済み node 集合 A に仮想 root 0 を加える。拡張スコア行列 S^+ を

$$S_{ij}^+ = \begin{cases} s_{ij}, & i, j \in A, \\ r_j - \lambda, & i = 0, j \in A, \\ r_i - \lambda, & j = 0, i \in A, \\ -\infty, & i = j \end{cases} \quad (44)$$

と定義する。無向 MST として解く場合、 S^+ は対称にする。

最大全域木を

$$T_{proj}^+ = \arg \max_{T^+ \in \mathcal{T}(G^+)} \sum_{(i,j) \in T^+} S_{ij}^+ \quad (45)$$

とする。実装で minimum spanning tree を使う場合は、cost を

$$C_{ij}^+ = -S_{ij}^+ \quad (46)$$

として MST を解けばよい。

9.2 projection mask

MST 結果から mask を作る。

$$m_{ij}^{proj} = \mathbf{1}[(i, j) \in T_{proj}^+], \quad i, j \in A \cup \{0\}. \quad (47)$$

実 edge に対する mask は m_{ij}^{proj} 、root edge に対する mask は m_{0i}^{proj} である。

9.3 SFS の一般形

SFS は、MST projection で選ばれなかった edge の edge logit を抑制し、選ばれた edge の non-edge logit を抑制する操作として定義できる。2-class logit を

$$\mathbf{z}_{ij} = (z_{ij}^{edge}, z_{ij}^{non}) \quad (48)$$

とする。projection mask に基づいて、

$$\tilde{z}_{ij}^{edge} = z_{ij}^{edge} - \gamma \mathbf{1}[m_{ij}^{proj} = 0], \quad (49)$$

$$\tilde{z}_{ij}^{non} = z_{ij}^{non} - \gamma \mathbf{1}[m_{ij}^{proj} = 1] \quad (50)$$

とする。 $\gamma > 0$ は suppression strength である。root edge にも同じ操作を適用する。

この定義は TreeFormer/PlantPose の SFS の思想と同じである。PlantPose の説明では、MST projection が追加すべき edge では non-edge feature を抑制し、削除すべき edge では edge feature を抑制することで、MST 構造に整合する edge existence を出しつつ勾配を流す、とされている [2]。

9.4 Virtual-Root SFS loss

SFS 後の logit に対して、拡張 GT label y_{ij}^+ を定義する。

$$y_{ij}^+ = \begin{cases} y_{ij}^*, & i, j \in A, \\ y_{0i}^*, & i = 0, j = i \in A, \\ 0, & \text{otherwise.} \end{cases} \quad (51)$$

root 代表が任意の場合、 y_{0i}^* は固定せず、MIL root loss を併用する。この場合、 $\mathcal{L}_{VR-SFS}$ は実 edge にのみ適用してもよい。

固定 root がある場合の VR-SFS loss は、

$$\mathcal{L}_{VR-SFS} = \frac{1}{|\mathcal{P}^+|} \sum_{(i,j) \in \mathcal{P}^+} \text{CE}(\text{softmax}(\tilde{\mathbf{z}}_{ij}), y_{ij}^+). \quad (52)$$

$\mathcal{P}^+$ は実 edge 候補と root edge 候補の和集合である。

注意 1. VR-SFS は「MST を完全に微分可能化する」方法ではない。非微分な MST の結果を使って、*logit* または *feature* を選択的に抑制し、*edge loss* の *forward* 値を構造制約に寄せる方法である。厳密な *structured probabilistic loss* が必要なら、次節の *Rooted Forest CRF* を使う。

10 Rooted Forest CRF: より上位互換に近い定式化

10.1 有向仮想ルートグラフ

より理論的に整理するなら、仮想ルート 0 を根とする有向全域木、すなわち *arborescence* として扱う。各実 node はちょうど 1 つの *incoming arc* を持ち、root 0 から全 node へ到達可能である。root arc $0 \rightarrow i$ は component 開始を表し、実 arc $i \rightarrow j$ は component 内の親子関係を表す。

スコアを

$$\theta_{ij} = \begin{cases} s_{ij}^{dir}, & i, j \in V, \\ r_j - \lambda, & i = 0, j \in V, \\ -\infty, & j = 0 \end{cases} \quad (53)$$

とする。重みは

$$w_{ij} = \exp(\theta_{ij}/\tau) \quad (54)$$

である。 $\tau > 0$ は temperature である。

10.2 Matrix-Tree partition

有向 *arborescence* に対する partition function は Matrix-Tree Theorem で計算できる。Koo et al. は、directed spanning tree を含む structured prediction で、Kirchhoff の Matrix-Tree Theorem を用いて partition function と marginal を計算する枠組みを示している [3]。

ここでは、arc $i \rightarrow j$ の weight を w_{ij} とし、incoming Laplacian を

$$L_{jj} = \sum_{i \neq j} w_{ij}, \quad L_{ij} = -w_{ij} \quad (i \neq j) \quad (55)$$

と定義する。root 0 に対応する行・列を削除した小行列を $L_{\bar{0}}$ とすると、

$$Z(\theta) = \det L_{\bar{0}} = \sum_{T^+ \in \mathcal{A}_0(G^+)} \prod_{(i \rightarrow j) \in T^+} w_{ij} \quad (56)$$

である。 $\mathcal{A}_0(G^+)$ は root 0 から外向きに向いた有向全域木集合である。

10.3 root 固定の NLL

正解の拡張有向木 T^{+*} が一意に定まる場合、negative log-likelihood は

$$\mathcal{L}_{VR-CRF}^{fixed} = - \sum_{(i \rightarrow j) \in T^{+*}} \log w_{ij} + \log Z(\theta). \quad (57)$$

これは edge を独立に扱う BCE ではなく、全ての rooted forest 候補に対する正規化を持つ structured loss である。

10.4 root 未指定の場合の周辺化

component root が任意でよい場合、正解森林 F^* に整合する全ての root 選択を周辺化する。component C_c^* ごとに root 候補 $\rho_c \in C_c^*$ を選び、各 component 内の GT edge を ρ_c から外向きに向ける。このようにして得られる拡張有向木集合を

$$\mathcal{A}(F^*) \quad (58)$$

とする。

正解森林の周辺尤度は

$$P(F^* | I) = \frac{Z_{F^*}(\theta)}{Z(\theta)}, \quad (59)$$

ただし

$$Z_{F^*}(\theta) = \sum_{T^+ \in \mathcal{A}(F^*)} \prod_{(i \rightarrow j) \in T^+} w_{ij}. \quad (60)$$

よって loss は

$$\mathcal{L}_{VR-CRF}^{marginal} = -\log Z_{F^*}(\theta) + \log Z(\theta). \quad (61)$$

この形式は、root 選択に任意性がある植物骨格に適している。特定 node を root に固定せず、「各 component に 1 つ root edge がある」という構造だけを学習できる。

10.5 Matrix-Forest との関係

無向グラフに対しては、rooted forests を Laplacian の行列式で数える Matrix-Forest 型の定理が知られている。例えば、rooted forests を component 数で数える生成関数が graph Laplacian を用いて表現できることが示されている [4]。本資料の仮想ルート CRF は、実装上は有向 Matrix-Tree Theorem を使う方が扱いやすいが、理論的には Matrix-Forest 型の定式化と同じ問題族に属する。

11 Perturbed Spanning Forest loss

MST/MSF は離散最適化であり、通常の意味では微分不能である。近年は、離散 optimizer に摂動を加えて微分可能な近似を作る方法が研究されている。Differentiable Clustering with Perturbed Spanning Forests は、minimum-weight spanning forests に確率的摂動を入れ、end-to-end trainable な clustering operation として使う方法を提案している [5]。また、black-box combinatorial solver へ効率的な backward pass を与える研究もある [6]。

TreeFormer の仮想ルート問題に応用するなら、スコア θ にノイズ ϵ を足し、

$$Y_\epsilon(\theta) = \text{MSF}(\theta + \epsilon) \quad (62)$$

を複数回計算する。期待 projection

$$\bar{Y}(\theta) = \mathbb{E}_\epsilon[Y_\epsilon(\theta)] \quad (63)$$

を使い、GT forest indicator Y^* との距離を取る。

$$\mathcal{L}_{\text{PerturbedForest}} = \|\bar{Y}(\theta) - Y^*\|_2^2 \quad \text{or} \quad \mathcal{L}_{FY}(\theta; Y^*). \quad (64)$$

この方法は、仮想ルート付き MST/SFS より研究実装寄りである。利点は、MST/MSF そのものの選択を勾配近似に含められることである。欠点は、実装・計算量・再現性管理が難しくなることである。

12 GNN/adjacency loss との関係

12.1 GNN loss でできること

GNN 的な loss や adjacency reconstruction loss は、node embedding を改善し、edge classifier を強くするために有用である。例えば、

$$\hat{A}_{ij} = \text{sigmoid}(g(z_i, z_j)) \quad (65)$$

として、

$$\mathcal{L}_{adj} = \text{BCE}(\hat{A}, A^*) \quad (66)$$

を追加する。message passing により、局所 pair だけでなく周辺 node の文脈を使える。

12.2 GNN loss でできないこと

しかし、GNN loss は通常、以下を直接保証しない。

$$\text{acyclicity, component count, exact edge cardinality, no cross-component bridge.} \quad (67)$$

最終出力が pairwise adjacency probability であれば、threshold 後に cycle、孤立 component、余分な bridge が出る可能性は残る。したがって、GNN は主 loss ではなく、edge score の品質を上げる補助 loss として使うのが妥当である。

12.3 推奨する併用

実用上は以下がよい。

$$\mathcal{L}_{total} = \mathcal{L}_{node} + \lambda_{edge}\mathcal{L}_{edge} + \lambda_{gnn}\mathcal{L}_{adj/GNN} + \lambda_{root}\mathcal{L}_{root} + \lambda_{cross}\mathcal{L}_{cross} + \lambda_{vr}\mathcal{L}_{VR-SFS}. \quad (68)$$

GNN で edge score を改善し、仮想ルート MST/SFS で構造制約をかける、という分担である。

13 推論アルゴリズム

13.1 component 数が未知の場合

component 数が未知の場合は、root penalty λ を使って拡張 MST を解く。

入力: active node 集合 A 、実 edge score s_{ij} 、root score r_i 、penalty λ 。出力: 予測森林 F 、root edge 集合 R 、component ID。

1. 拡張 node 集合 $A^+ = A \cup \{0\}$ を作る。
2. 実 edge (i, j) に score s_{ij} を与える。
3. root edge $(0, i)$ に score $r_i - \lambda$ を与える。
4. 最大全域木 T^+ を解く。実装で MST を使う場合は $\text{cost} - \text{score}$ を使う。
5. root edge 集合 $R = T^+ \cap E_0$ を取り出す。
6. $F = T^+ \setminus R$ を予測森林とする。
7. F の connected components を計算し、component ID を付与する。

このとき、component 数は $|R| = \deg_{T^+}(0)$ である。

13.2 component 数が既知の場合

画像ごとに component 数 C が既知なら、root degree を C に近づける。実装方法は 3 つある。

方法	内容	推奨度
λ 探索	λ を変えながら拡張 MST を解き、 $\deg(0) = C$ になる解を探す	実装容易。ただし必ず一致する保証はない
Kruskal 停止	実 node 間で maximum spanning forest を作り、edge 数が $N - C$ になったら停止。各 component の root は最大 r_i で選ぶ	root score の影響を分離してよければ安定
degree-constrained MST	$\deg(0) = C$ を制約に入れて最適化する	厳密だが実装が重い

短期実装では、 λ 探索または Kruskal 停止で十分である。root score を component partition にも影響させたいなら、 λ 探索を優先する。

13.3 node 数も未知の場合

実運用では node 数 N も未知である。モデルは最大 K 個の候補を出す。

$$q_i = P(\text{node } i \text{ exists}). \quad (69)$$

active node 集合は、

$$A = \{i : q_i > \tau_{\text{node}}\} \quad (70)$$

または $\text{top-}N_{\max}$ で決める。仮想ルート MST は active node 集合上でのみ解く。低 confidence node を含めると、root edge や bridge edge が不安定になるため、node threshold と root penalty は同時に validation する必要がある。

14 ONNX/OpenVINO を見据えた出力契約

仮想ルート方式は、モデル本体を固定テンソル出力に保てる。推奨出力は以下である。

$$\text{node_scores} \in \mathbb{R}^{B \times K}, \quad (71)$$

$$\text{node_xy} \in \mathbb{R}^{B \times K \times 2}, \quad (72)$$

$$\text{edge_scores} \in \mathbb{R}^{B \times P}, \quad P = K(K-1)/2, \quad (73)$$

$$\text{root_scores} \in \mathbb{R}^{B \times K}, \quad (74)$$

$$\text{count_logits} \in \mathbb{R}^{B \times (C_{\max}+1)} \quad \text{optional}. \quad (75)$$

edge pair table は定数として外部に保持する。

$$\text{edge_pairs} \in \mathbb{N}^{P \times 2}. \quad (76)$$

ONNX/OpenVINO モデルは graph object を出さない。仮想ルート MST、root edge 削除、component ID 付与、MST 後の枝刈り、可視化、JSON 変換はモデル外後処理にする。

この分離により、モデルはデプロイしやすい固定テンソル推論に集中できる。離散グラフ処理は C++/Python 側の小さな後処理として実装する。

15 実装仕様案

15.1 forward 出力

モデル forward は以下を返す。

```
{
  "node_logits": FloatTensor[B, K, 2],
  "node_xy":      FloatTensor[B, K, 2],
  "edge_logits": FloatTensor[B, P, 2],
  "root_logits": FloatTensor[B, K],
  "count_logits": Optional[FloatTensor[B, Cmax + 1]]
}
```

15.2 postprocessor 出力

後処理は以下を返す。

```
{
  "nodes":          FloatTensor[N, 2],
  "edges":          LongTensor[M, 2],
  "root_edges":     LongTensor[C, 2],  # (0, node_id)
  "component_id":   LongTensor[N],
  "node_scores":    FloatTensor[N],
  "edge_scores":    FloatTensor[M]
}
```

ここで C は予測 component 数、 $M = N - C$ が森林の edge 数である。予測後に枝刈りを行う場合、 $M \leq N - C$ になることがあるため、その場合は「木制約を保つ後処理」か「森林から部分森林を許す後処理」かを明示する。

15.3 loss module 入力

学習時の loss module は以下を受け取る。

```
pred = {
  node_logits, node_xy, edge_logits, root_logits, count_logits
}
target = {
  gt_nodes, gt_edges, gt_component_id, optional_gt_roots, optional_count
}
match = HungarianMatch(pred.nodes, target.gt_nodes)
```

15.4 実装上の重要パラメータ

記号/名前	意味	推奨する決め方
-------	----	---------

K	最大 node 候補数	dataset の 99 percentile 以上
P	edge 候補数	全 pair なら $K(K-1)/2$
λ	root edge penalty	validation で tree/forest F1 と component error を見て調整
γ	SFS suppression strength	既存 TreeFormer の設定を初期値にし、sweep する
τ_{node}	node threshold	node recall 重視でやや低めから調整
α	edge positive weight	positive/negative imbalance から初期化
λ_{cross}	component 間 bridge 抑制重み	bridge 誤検出が多い場合に上げる

16 評価指標

非連結対応では、通常の node/edge 指標だけでは不十分である。最低限、以下を測る。

指標	意味
node precision/recall	node 検出そのものの品質
coordinate error	node 位置誤差
edge precision/recall/F1	実 edge の分類品質
component count error	$ \hat{C} - C^* $
bridge false positive rate	異なる GT component 間に予測 edge を張った割合
forest validity rate	予測 graph が無閉路かつ各 component 内で木になっている割合
cycle count	閉路数
connected component F1	component 分割の一致度
SMD/TOPO	TreeFormer/PlantPose 系の既存評価に合わせた構造類似度
post-MST edge F1 latency	仮想ルート MST 後の edge 品質 model 推論と postprocess を分けて測る

17 想定される失敗と対処

17.1 root edge が安すぎる場合

root edge penalty λ が小さすぎる、または root score が過大な場合、各 node が root へ直接つながり、細かく分断された forest になる。

$$\deg(0) \approx N. \quad (77)$$

対処は、 λ を上げる、root count loss を強める、root 候補を component 内で 1 つに絞る loss を入れることである。

17.2 root edge が高すぎる場合

λ が大きすぎる場合、モデルは root edge を避け、別 component 間の実 edge を使って単一木に近づく。

$$\deg(0) \approx 1. \quad (78)$$

対処は、 λ を下げる、cross-component negative loss を強める、component count loss を追加することである。

17.3 root が任意なのに固定 root CE を使う場合

component root に物理的意味がない場合、特定 node を root 正解に固定すると、不要なバイアスが入る。対処は MIL root loss または Rooted Forest CRF の root 周辺化を使うことである。

17.4 遮蔽と別個体を混同する場合

GT 仕様で「遮蔽された 1 本の木をつなぐべきか、見えている部分だけを forest として扱うか」を明確化する。評価指標もこれに合わせる。

18 実装検証用の最小テスト

実装時には以下の単体テストを必須にする。

テスト名	内容	期待結果
vr-two-trees	2 つの独立した 3-node tree に高 edge score、component 間に低 score を与える	root edge 2 本、実 edge 4 本、bridge 0 本
root-degree-components	任意の拡張 MST から root edge を削除して component 数を数える	component 数が root 次数と一致
lambda-under-split	λ を極端に小さくする	component 数が増える方向に変化
lambda-over-merge	λ を極端に大きくする	component 数が減る方向に変化
cross-loss-bridge	component 間 edge score だけを上げる	$\mathcal{L}_{cross}$ が増加

root-mil-any-node	component 内の任意 1 node だけ root score を上げる	MIL root loss が低下
forest-validity	postprocessor 出力に cycle がないか検査	cycle count 0
onnx-contract	出力 shape が固定契約を満たすか検査	node/edge/root score shape が一致

19 研究候補の比較

方式	長所	短所	推奨
Virtual-Root MST/SFS	既存 TreeFormer に最も近く、実装が軽い。component 数も root 次数で読める。	root penalty 較正が必要。CRF ほど確率的に綺麗ではない。	短期本命
Rooted Forest CRF	森林全体の尤度を直接最大化でき、root 周辺化も可能。	determinant 計算、数値安定性、実装がやや重い。	中期本命
Perturbed MSF	離散 forest solver 自体への勾配近似が可能。弱教師ありにも拡張しやすい。	Monte Carlo 計算、再現性、実装複雑性が課題。	研究比較
GNN + edge BCE	実装が簡単で edge score 改善に有効。	forest 制約を保証しない。bridge/cycle 対策が弱い。	補助 loss
Graph OT/PMFGW	可変 node graph 全体の距離を扱える。	tree/forest 制約を直接保証しない。計算量も課題。	補助 loss

20 最終提案

本資料の最終提案は、TreeFormer を以下のように拡張することである。

$$\boxed{\text{TreeFormer} + \text{root head} + \text{Virtual-Root MST/SFS} + \text{cross-component loss}} \quad (79)$$

具体的には、モデルは固定テンソルとして

$$(\text{node_scores}, \text{node_xy}, \text{edge_scores}, \text{root_scores}) \quad (80)$$

を出す。訓練時は、仮想ルート付き MST を使って SFS を行い、root loss と cross-component loss を併用する。推論時は、同じ score から拡張 MST を解き、root edge を削除して forest を得る。

この方法は、単一 MST の「必ず全 node を 1 本につなぐ」という問題を解消し、画像内に 2 本以上の別個の木が存在する場合にも自然に対応できる。さらに、出力を固定テンソルに保てるため、ONNX/OpenVINO を見据えたデプロイ設計とも整合する。

21 補足: 最小実装の疑似手順

21.1 訓練時

1. 画像 I をモデルへ入力する。
2. node logits、node xy、edge logits、root logits を得る。
3. Hungarian matching で GT node と予測 node を対応させる。
4. GT edge label、GT component label、cross-component negative pair を作る。
5. 対応済み node 集合 A 上に仮想 root 0 を追加する。
6. 実 edge scores s_{ij} と root score $r_i - \lambda$ で拡張スコア行列 S^+ を作る。
7. 最大全域木 T^+ を解く。
8. T^+ から projection mask を作り、edge/root logits に SFS を適用する。
9. $\mathcal{L}_{node}$ 、 $\mathcal{L}_{edge}$ 、 $\mathcal{L}_{root}$ 、 $\mathcal{L}_{cross}$ 、 $\mathcal{L}_{VR-SFS}$ を合算する。
10. backpropagation する。

21.2 推論時

1. node score threshold または top- K' で active node 集合 A を決める。
2. edge score と root score を抽出する。
3. root penalty λ を適用し、拡張グラフ G^+ を作る。
4. 最大全域木 T^+ を解く。
5. root edge を削除し、実ノード上の森林 F を得る。
6. connected components を計算し、component ID を付ける。
7. 必要に応じて短 edge/低 confidence edge の枝刈り、degree-2 node 整理、可視化を行う。

22 結語

仮想ルートノードは、TreeFormer の単一 MST 制約を非連結な複数木へ拡張するうえで、最も自然で実装可能性の高い定式化である。数学的には、仮想ルート付き全域木と root 付き森林が対応するため、root edge を除去するだけで実ノード上の森林が得られる。実装上は、既存の MST/SFS を大きく壊さず、root head、root penalty、cross-component loss を追加すればよい。

したがって、次に実装するなら、まず Virtual-Root MST/SFS を baseline として構築し、その後、Rooted Forest CRF または Perturbed Spanning Forest loss を上位互換候補として比較するのが妥当である。

References

- [1] Xinpeng Liu, Hiroaki Santo, Yosuke Toda, and Fumio Okura. TreeFormer: Single-View Plant Skeleton Estimation via Tree-Constrained Graph Generation. WACV 2025. https://openaccess.thecvf.com/content/WACV2025/papers/Liu_TreeFormer_Single-View_Plant_Skeleton_Estimation_via_Tree-Constrained_Graph_Generation_WACV_2025_paper.pdf
- [2] Xinpeng Liu, Hiroaki Santo, Yosuke Toda, and Fumio Okura. PlantPose: Universal Plant Skeleton Estimation via Tree-constrained Graph Generation. International Journal of Computer Vision, 2026. <https://link.springer.com/article/10.1007/s11263-026-02882-4>
- [3] Terry Koo, Amir Globerson, Xavier Carreras, and Michael Collins. Structured Prediction Models via the Matrix-Tree Theorem. EMNLP-CoNLL 2007. <https://aclanthology.org/D07-1015.pdf>
- [4] Olivier Bernardi and Caroline J. Klivans. Directed Rooted Forests in Higher Dimension. <https://www.dam.brown.edu/people/cklivans/Forests.pdf>
- [5] Lawrence Stewart, Francis Bach, Felipe Llinares-López, and Quentin Berthet. Differentiable Clustering with Perturbed Spanning Forests. NeurIPS 2023. https://proceedings.neurips.cc/paper_files/paper/2023/hash/637a456d89289769ac1ab29617ef7213-Abstract-Conference.html
- [6] Marin Vlastelica, Anselm Paulus, Vít Musil, Georg Martius, and Michal Rolínek. Differentiation of Blackbox Combinatorial Solvers. ICLR 2020. <https://openreview.net/forum?id=BkevoJSYPB>